Task 02 — Understand Python Data Types

Topics

- `int`
- `float`
- `str`
- `bool`
- Basic data types
- `type()`

Task

Create variables using the following data types:

- Integer
- Decimal number
- String
- Boolean

Print each value and its data type.

Then create 5 more variables of your choice and identify their data types.

Reference

Python data types: https://www.w3schools.com/python/python_datatypes.asp

Python `type()`: https://www.w3schools.com/python/ref_func_type.asp

---

Task 03 — Student Marks Calculator

Topics

- Variables
- User input
- Type conversion
- Arithmetic operators

Task

Create a program that asks the user to enter marks for:

- Python
- Mathematics
- Database
- Statistics
- Computer Science

Calculate and display:

- Total marks
- Average marks
- Percentage

Assume every subject is out of 100.

Reference

Python user input: https://www.w3schools.com/python/python_user_input.asp

Python type conversion: https://www.w3schools.com/python/python_casting.asp

Python operators: https://www.w3schools.com/python/python_operators.asp

---

Task 04 — Simple Calculator

Topics

- Input
- Operators
- Type conversion
- Output

Task

Create a calculator that asks the user for two numbers.

Perform:

- Addition
- Subtraction
- Multiplication
- Division
- Modulus

Display all results clearly.

Also try the program with decimal numbers.

Reference

Python arithmetic operators: https://www.w3schools.com/python/python_operators.asp

Python numbers: https://www.w3schools.com/python/python_numbers.asp

---

Task 05 — Type Conversion Practice

Topics

- `int()`

- `float()`
- `str()`
- Type conversion

Task

Take the following values from the user:

- Age
- Height
- Marks

Convert them into the correct data types.

Display the values and their types.

Then calculate the user's age after 5 years.

Reference

Python casting: https://www.w3schools.com/python/python_casting.asp

Python user input: https://www.w3schools.com/python/python_user_input.asp

---

Task 06 — Student Name Analyzer

Topics

- Strings
- String methods
- String indexing

Task

Ask the user to enter their full name.

Display:

- Full name
- Name in uppercase
- Name in lowercase
- Number of characters
- First character
- Last character

Also remove any unnecessary spaces from the beginning and end of the name.

Reference

Python strings: https://www.w3schools.com/python/python_strings.asp

Python string methods: https://www.w3schools.com/python/python_strings_methods.asp

---

Task 07 — Student Introduction

Topics

- Strings
- f-strings
- Variables

Task

Take the following information from the user:

- Name
- Age
- Course
- College
- City

Create a short introduction using an f-string.

Example

```
Hello, my name is Rahul.
I am 20 years old and I am studying Data Science at ABC College.
I live in Surat.
```

Reference

Python string formatting: https://www.w3schools.com/python/python_string_formatting.asp

---

Task 08 — Pass or Fail

Topics

- `if`
- `else`
- Comparison operators

Task

Ask the user to enter their percentage.

If the percentage is 40 or above, display:

```
Result: Pass
```

Otherwise:

```
Result: Fail
```

Also handle values below 0 and above 100.

Reference

Python conditions: https://www.w3schools.com/python/python_conditions.asp

Python comparison operators: https://www.w3schools.com/python/python_operators.asp

---

Task 09 — Student Grading System

Topics

- `if`
- `elif`
- `else`
- Comparison operators

Task

Create a grading system based on percentage.

Use:

```
90 - 100  → A+
80 - 89   → A
70 - 79   → B
60 - 69   → C
50 - 59   → D
40 - 49   → E
Below 40  → Fail
```

Display the grade according to the entered percentage.

Also make sure invalid percentages are rejected.

Reference

Python if...elif...else: https://www.w3schools.com/python/python_conditions.asp

---

Task 10 — Mini Project: Student Result Calculator

Topics

- Variables
- Input
- Type conversion
- Operators
- Strings
- Conditions

Task

Create a complete student result program.

The program should ask for:

- Student name
- Roll number
- Marks of 5 subjects

Calculate:

- Total
- Percentage
- Grade
- Pass/Fail

Display a properly formatted result.

Reference

Review:

https://www.w3schools.com/python/python_variables.asp

https://www.w3schools.com/python/python_user_input.asp

https://www.w3schools.com/python/python_operators.asp

https://www.w3schools.com/python/python_conditions.asp

LEVEL 2 — LOOPS AND PROBLEM SOLVING

Goal

In this level you will learn how to repeat operations and process multiple values.

You will work with:

- `for` loops

- `while` loops
- `range()`
- `break`
- `continue`
- Basic numerical problem solving

Task 11 — Print Numbers

Topics

- `for` loop
- `range()`

Task

Write programs to:

1. Print numbers from 1 to 100.
2. Print numbers from 100 to 1.
3. Print numbers from 1 to 50.
4. Print numbers from 50 to 100.

Reference

Python for loops: https://www.w3schools.com/python/python_for_loops.asp

Python `range()`: https://www.w3schools.com/python/ref_func_range.asp

---

Task 12 — Even and Odd Numbers

Topics

- Loops
- Modulus operator
- Conditions

Task

Print all even numbers between 1 and 100.

Then create another program that prints all odd numbers between 1 and 100.

Reference

Python operators: https://www.w3schools.com/python/python_operators.asp

Python for loops: https://www.w3schools.com/python/python_for_loops.asp

Python conditions: https://www.w3schools.com/python/python_conditions.asp

---

Task 13 — Multiplication Table

Topics

- `for` loop
- `range()`
- Arithmetic operators

Task

Ask the user to enter a number.

Print its multiplication table from 1 to 10.

Example

```
Enter number: 7

7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Reference

Python for loops: https://www.w3schools.com/python/python_for_loops.asp

---

Task 14 — Sum of Numbers

Topics

- Loops
- Variables
- Accumulation

Task

Calculate the sum of numbers from 1 to 100.

Then modify the program so the user can enter the ending number.

For example:

```
Enter ending number: 50
```

The program should calculate the sum from 1 to 50.

Reference

Python for loops: https://www.w3schools.com/python/python_for_loops.asp

---

Task 15 — Factorial

Topics

- Loops
- Multiplication
- User input

Task

Ask the user for a number and calculate its factorial.

Example:

```
5! = 120
```

Try the program with different numbers.

Reference

Python for loops: https://www.w3schools.com/python/python_for_loops.asp

---

Task 16 — Prime Number Checker

Topics

- Loops
- Conditions
- Modulus

Task

Ask the user for a number and check whether it is prime.

Example:

```
Enter number: 17
17 is a prime number.
```

Also test:

- 2

- 7
- 10
- 15
- 25

Reference

Python conditions: https://www.w3schools.com/python/python_conditions.asp

Python for loops: https://www.w3schools.com/python/python_for_loops.asp

---

Task 17 — Find Maximum and Minimum

Topics

- Loops
- Conditions
- Comparison

Task

Create a program that takes 10 numbers from the user.

Find:

- Largest number
- Smallest number

First try solving it using loops and conditions.

After that, check how Python's built-in `max()` and `min()` functions work.

Reference

Python for loops: https://www.w3schools.com/python/python_for_loops.asp

Python built-in functions: https://www.w3schools.com/python/python_ref_functions.asp

---

Task 18 — Number Counter

Topics

- Loops
- Conditions
- Counters

Task

Take 10 numbers from the user.

Count how many are:

- Positive
- Negative
- Zero
- Even
- Odd

Display the final counts.

Reference

Python while loops: https://www.w3schools.com/python/python_while_loops.asp

Python conditions: https://www.w3schools.com/python/python_conditions.asp

---

Task 19 — Pattern Practice

Topics

- Nested loops
- `range()`

Task

Create the following patterns:

```
*
**
***
****
*****
```

Then:

```
1
12
123
1234
12345
```

Then try creating your own pattern.

Reference

Python for loops: https://www.w3schools.com/python/python_for_loops.asp

Python nested loops: https://www.w3schools.com/python/python_for_loops.asp

---

Task 20 — Mini Project: Number Analysis Program

Topics

- Loops
- Conditions
- Operators
- Counters
- Basic problem solving

Task

Ask the user how many numbers they want to enter.

Then accept the numbers and calculate:

- Total
- Average
- Maximum
- Minimum
- Number of even values
- Number of odd values
- Number of positive values
- Number of negative values

Display all results clearly.

Reference

Review:

https://www.w3schools.com/python/python_for_loops.asp

https://www.w3schools.com/python/python_conditions.asp

https://www.w3schools.com/python/python_operators.asp

LEVEL 3 — LISTS, TUPLES, SETS AND DICTIONARIES

Goal

This is an important level for Data Science.

Real datasets contain collections of values. Before working with NumPy and Pandas, you should understand how Python stores and works with multiple values.

You will learn:

- Lists
- List methods

- List indexing and slicing
- Tuples
- Sets
- Dictionaries
- Looping through collections
- Basic data organization

Part A — Lists

Task 21 — Student Marks List

Topics

- Lists
- Indexing
- `len()`

Task

Create a list containing marks of 10 students.

Display:

- Complete list
- First mark
- Last mark
- Number of students

Then change one student's marks.

Reference

Python lists: https://www.w3schools.com/python/python_lists.asp

Python list indexing: https://www.w3schools.com/python/python_lists_access.asp

---

Task 22 — List Methods Practice

Topics

- `append()`
- `insert()`
- `remove()`
- `pop()`
- `sort()`
- `reverse()`

Task

Create a list of 5 student names.

Practice each of the following:

1. Add a new student.
2. Insert a student at a specific position.
3. Remove a student.
4. Remove the last student.
5. Sort the names.
6. Reverse the list.

Print the list after each operation.

Reference

Python list methods: https://www.w3schools.com/python/python_lists_methods.asp

---

Task 23 — Calculate Marks from a List

Topics

- Lists
- Loops
- `sum()`
- `len()`

Task

Create a list of student marks.

Calculate:

- Total marks
- Number of subjects
- Average marks
- Highest mark
- Lowest mark

Try to first solve the problem using a loop.

Then check Python's built-in functions.

Reference

Python lists: https://www.w3schools.com/python/python_lists.asp

Python built-in functions: https://www.w3schools.com/python/python_ref_functions.asp

---

Task 24 — Filter Marks

Topics

- Lists
- Loops
- Conditions

Task

Given a list of marks, create separate lists for:

- Marks greater than or equal to 70
- Marks below 70
- Passing marks
- Failing marks

Reference

Python list access: https://www.w3schools.com/python/python_lists_access.asp

Python conditions: https://www.w3schools.com/python/python_conditions.asp

---

Task 25 — List Comprehension

Topics

- List comprehension
- Conditions

Task

Create lists containing:

1. Squares of numbers from 1 to 10.
2. Even numbers from 1 to 50.
3. Numbers greater than 50 from a given list.
4. Names starting with the letter A.

Reference

Python list comprehension: https://www.w3schools.com/python/python_lists_comprehension.asp

---

Part B — Tuples

Task 26 — Student Tuple

Topics

- Tuples
- Indexing

- Tuple unpacking

Task

Create a tuple containing:

```
Name
Age
Course
City
```

Display each value separately.

Then try changing one value and observe what happens.

Reference

Python tuples: https://www.w3schools.com/python/python_tuples.asp

Python tuple access: https://www.w3schools.com/python/python_tuples_access.asp

---

Part C — Sets

Task 27 — Common Students

Topics

- Sets
- Union
- Intersection
- Difference

Task

Create two sets:

```
students_python
students_data_science
```

Store student names in both.

Find:

- Students learning both subjects
- Students learning only Python
- Students learning only Data Science

- All unique students

Reference

Python sets: https://www.w3schools.com/python/python_sets.asp

Python set methods: https://www.w3schools.com/python/python_sets_methods.asp

---

Part D — Dictionaries

Task 28 — Student Information Dictionary

Topics

- Dictionaries
- Keys
- Values
- Accessing values

Task

Create a dictionary containing information about one student:

```
name
age
course
city
percentage
```

Display every value separately.

Then update the percentage and add a new field called `email`.

Reference

Python dictionaries: https://www.w3schools.com/python/python_dictionaries.asp

Access dictionary items: https://www.w3schools.com/python/python_dictionaries_access.asp

---

Task 29 — Multiple Student Records

Topics

- Dictionaries
- Lists
- Loops

Task

Create records for at least 5 students.

Each student should have:

- Name
- Age
- Course
- Marks

Store the information using a suitable Python data structure.

Then display:

- All students
- Student with highest marks
- Student with lowest marks
- Average marks

Reference

Python dictionaries: https://www.w3schools.com/python/python_dictionaries.asp

Python lists: https://www.w3schools.com/python/python_lists.asp

Python loops: https://www.w3schools.com/python/python_for_loops.asp

---

Task 30 — Mini Project: Student Data Management

Topics

- Lists
- Dictionaries
- Loops
- Conditions
- List methods

Task

Create a simple student data management program.

The program should allow the user to:

1. Add a student.
2. View all students.
3. Search for a student.
4. Update student marks.
5. Delete a student.
6. Find the student with the highest marks.
7. Calculate average marks.

Each student should contain:

- Name
- Age
- Course
- Marks

Use a list of dictionaries or another suitable structure.

Reference

Python dictionaries: https://www.w3schools.com/python/python_dictionaries.asp

Python lists: https://www.w3schools.com/python/python_lists.asp

Python list methods: https://www.w3schools.com/python/python_lists_methods.asp

Python conditions: https://www.w3schools.com/python/python_conditions.asp

Python for loops: https://www.w3schools.com/python/python_for_loops.asp

---

LEVEL 3 CHECKPOINT

Before moving to the next level, you should be able to answer these questions yourself:

Python Basics

- What is a variable?
- What is the difference between `int`, `float` and `str`?
- Why do we use `input()`?
- Why do we convert input using `int()` or `float()`?
- What is the difference between `=` and `==`?
- What is the purpose of `if`, `elif` and `else`?

Loops

- Why do we use loops?
- What is the difference between `for` and `while`?
- What does `range()` do?
- When would you use `break`?
- When would you use `continue`?

Collections

- What is a list?
- What is a tuple?
- What is a set?
- What is a dictionary?
- When would you use a list instead of a tuple?

- What is the difference between a list and a set?
- How do you access a value from a dictionary?
- How do you loop through a list?
- How do you loop through a dictionary?

Level 3 Final Practice

Do this task without looking at previous solutions.

Student Performance Analysis

Create a Python program that stores information for at least 10 students.

Each student should have:

- Name
- Age
- Course
- Marks

Your program should calculate:

1. Total number of students.
2. Average marks.
3. Highest marks.
4. Lowest marks.
5. Number of students who passed.
6. Number of students who failed.
7. Students who scored above 75.
8. Student with the highest marks.
9. Student with the lowest marks.
10. Display all student records.

Try to keep the program organized and easy to understand.

Reference

Use the references from Tasks 21–30.

Do not search for a complete solution. Use the documentation to understand the individual concepts and build the program yourself.

Before Starting NumPy

Do not move to NumPy just because all tasks are completed.

You should be comfortable with:

- Variables
- Input and output
- Conditions
- Loops
- Lists
- Tuples
- Sets
- Dictionaries
- Indexing
- Slicing
- Basic functions
- Basic problem solving

If you can write the Level 3 final practice program on your own and explain how it works, you are ready to move to the next stage.
